



Cyber Effects Capstone



Pacebreakers



Amy Chang, Ayush Ravi Chandran, Joe Lebedev, Joyce Werhane

Environment and Assumptions

Target Environment

- Our target is an office network that utilizes a Samba file server to share files between systems.
- We aim to infiltrate the system and obtain important files and credentials.

Assumptions

- We assume systems to be using Ubuntu 16.04 or similar releases
- We assume the Samba server to be version 4.6.4 or earlier

Vulnerability

CVE-2017-7494 - SambaCry

- Samba versions before 4.6.4 on Linux systems with writable shares enabled
- Samba fails to properly validate Windows Named Pipe paths, allowing path traversal to arbitrary locations on the filesystem
- Samba typically runs with elevated privileges on Linux, so any code it loads inherits those privileges

Trigger: a writable share must exist and the attacker must be an authenticated client on the network

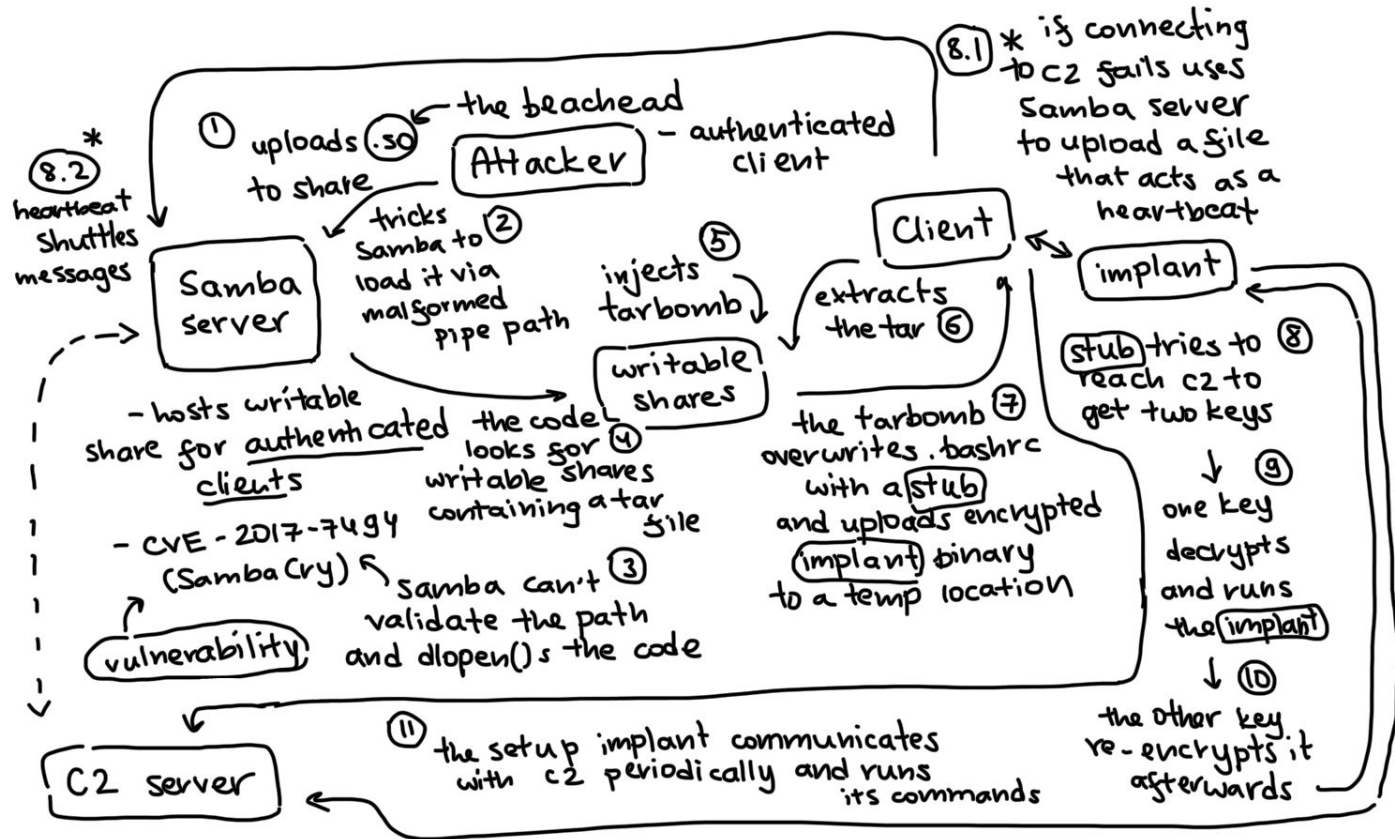
Exploit

- Attacker must be an authenticated client on the network with access to a writable Samba share
- Attacker uploads a malicious .so (shared library) to their writable share
- Attacker sends a request with a Windows Named Pipe path that traverses to the uploaded .so
- Vulnerable Samba versions don't validate these pipe paths, so the server follows the traversal
- Samba calls dlopen() on the .so, loading and executing the attacker's code with Samba's privileges
- The .so code runs on the Samba server and sets up the rest of the chain

Steps

1. Attacker uploads malicious .so file to Samba server, has the server execute it through SambaCry, and runs privilege escalation.
2. Samba server implant finds a writable share path with a tar, and injects it with tar entries
3. Clients within the server extract the unassuming tarbomb, creating a bash stub and an encrypted binary implant.
4. The bash stub runs upon an interactive shell opening, attempting to connect with the C2 server to receive keys to decrypt and re-encrypt the client's binary implant. Failed connections will send a hidden file to the share path.
5. C2 communication is established to the implants. Client implants that were unable to connect with HTTPS utilize properly communicating implants as gateways. The Samba server implant performs routing for all client implants.

Architecture Diagram



Beach-Heading Network

- Consists of a controlling server, a routing implant on the Samba server, and clients with writable shares on the Samba server.
- The Samba server implements a rudimentary node mesh of clients by dropping tar slips into tars on writable share.
- When a client extracts the tar file, the relative pathing of the entries overwrites `~./bashrc` with a basic stub to launch the implant.
- Once the client nodes are started, the Samba server performs routing between endpoints and gateways.

Privilege Escalation Race Condition

The privilege escalation works by targeting a race condition in `PTRACE_TRACEME`. When a child process calls the function, the kernel attempts to check the parent's credentials, a race condition occurs where they could be changed before the check is completed, based on collaboration with the parent.

Privilege Escalation & Persistence

Privilege Escalation

- Root privilege is gained through a ptrace function race condition

Persistence

- A custom init.d file is downloaded along with the implant, which tells the system to run our implant upon system boot.
- The file attempts to appear official, utilizing the start-stop-daemon and implementing options for stopping and restarting the service (though the stop command doesn't actually do anything)

```
init.d script
```

```
NAME=dbus-sync
```

```
DAEMON=/usr/bin/$NAME
```

```
do_start() {  
    start-stop-daemon --start  
    --quiet --background \  
        --exec $DAEMON ||  
return 2  
}
```

Implant

C2 Command Polling

Periodically checks for commands from C2 every **4-12 minutes**.

Capabilities:

- Run terminal commands
- Send target machine info
- Specified file exfiltration
- Exfil Firefox credentials & keys
- Exfil /etc/shadow password file
- Self-destruct functionality

Persistence

Separate file stored in `/etc/init.d` to ensure persistence through reboot.



Implant obfuscation

- Hidden in plain sight as a service called 'dbus-sync' in `/usr/bin`
- Written in python. Compiled as a stripped binary file using pyinstaller
- Upon self-destruct, init.d files, bash, sys, and auth logs are wiped of our "dbus-sync service"
- All information is sent to C2 through HTTPS
- Random intervals between C2 issued commands

C2 Architecture

Operator Communication

Operator → C2 :9998 (localhost only)

- Secure token-authenticated operator API
- Operator queues tasks per implant session

Implant Communication

Implant → Redirector :443 → C2 :9999

- Implant only knows the redirector IP so C2 is never exposed
- Redirector (nginx TCP stream proxy) can be swapped if burned
- Persistent open socket. C2 pushes commands down, no polling

Encryption

C2 channel (port 443)

- New RSA-2048 key pair generated per session (ephemeral) Sends public key to C2; C2 sends its public key back.
- Key exchange on connect. There are no static keys to extract from binary
- Messages: RSA-OAEP key wrap + Fernet (AES-128-CBC + HMAC-SHA256)
- XOR obfuscation layer on top

Exfil channel (port 9443)

- Separate channel to receive files
- Separate encryption scheme: rolling XOR + base64
- X-Enc: rxb64 header signals encoding to receiver

C2 Commands

HEARTBEAT	Keepalive check
SYSINFO	Hostname, OS, user, IP, uptime
RUN_CMD	Execute shell command, return stdout/stderr
RECON_BUNDLE	Full recon: processes, users, SUID bins, crontabs, env
EXFIL_FILE	Read any file and POST to exfil server
SHADOW_EXFIL	Exfil <code>/etc/shadow</code> (password hashes)
FIREFOX_EXFIL	Steal Firefox <code>key4.db</code> + <code>logins.json</code> (saved passwords)
DELETE_FILE	Remove a file on target
DESTROY	Self-destruct: removes binary, persistence, wipes logs

Evasion Techniques

Binary & Process

Compiled + stripped PyInstaller (no source/symbols).
Process `dbus-sync` blends with D-Bus infrastructure.

Network Identity

Port 443 (HTTPS lookalike). Spoofed TLS :

`CN=security.ubuntu.com`

`O=Canonical Ltd`

C2 Obfuscation

Redirector hop: implant only sees nginx IP. Can be hosted separately.

Jitter: random 4-12s beacons remove heartbeat signature.

Encryption & Cleanup

Ephemeral RSA keys per session. DESTROY command wipes binary, logs, history, and persistence.

DESTROY

```
WIPE BINARY
ERASE INIT.D
CLEAN CRONTAB
CLEAN BASH_HISTORY
REMOVE SYSLOG ENTRIES
```

Network Traffic Profile

- All traffic port 443, TLS encrypted.
- No plaintext visible
- Long idle gaps between bursts (4-12s jitter)
- Small burst only when command issued
- CPU: 0.1% load
- RAM: <1.5% load

tcp.port == 443							
No.	Time	Source	Destination	Protocol	Length	Info	
1459	26.462515509	10.0.2.15	172.29.80.1	TCP	74	48328 → 443	[SYN] S
1460	26.463644646	172.29.80.1	10.0.2.15	TCP	60	443 → 48328	[SYN, A
1461	26.463683499	10.0.2.15	172.29.80.1	TCP	54	48328 → 443	[ACK] S
1462	26.464210339	10.0.2.15	172.29.80.1	TLSv1.3	311	Client Hello	
1463	26.464594658	172.29.80.1	10.0.2.15	TCP	60	443 → 48328	[ACK] S
1464	26.468171685	172.29.80.1	10.0.2.15	TLSv1.3	1398	Server Hello, Chang	
1465	26.468191256	10.0.2.15	172.29.80.1	TCP	54	48328 → 443	[ACK] S
1466	26.468827388	10.0.2.15	172.29.80.1	TLSv1.3	134	Change Cipher Spec,	
1467	26.469052661	172.29.80.1	10.0.2.15	TCP	60	443 → 48328	[ACK] S
1468	26.471787993	172.29.80.1	10.0.2.15	TLSv1.3	309	Application Data	

Frame 1468: 309 bytes on wire (2472 bits), 309 bytes captured (2472 bits) on interface enp0s3, i

Interface id: 0 (enp0s3)

Encapsulation type: Ethernet (1)

Arrival Time: May 6, 2026 14:46:32.650147854 EDT

[Time shift for this packet: 0.000000000 seconds]

Epoch Time: 1778093192.650147854 seconds

[Time delta from previous captured frame: 0.002735332 seconds]

[Time delta from previous displayed frame: 0.002735332 seconds]

[Time since reference or first frame: 26.471787993 seconds]

Frame Number: 1468

Frame Length: 309 bytes (2472 bits)

Capture Length: 309 bytes (2472 bits)

0000	08 00 27 92 d5 bc 52 55	0a 00 02 02 08 00 45 00	..'....RU.....E..
0010	01 27 2e a6 00 00 40 06	42 fe ac 1d 50 01 0a 00	..'....@.B...P...
0020	02 0f 01 bb bc c8 01 d8	ad 42 d3 35 95 3d 50 18	B.5..P...
0030	ff ff b7 d8 00 00 17 03	03 00 fa a0 c7 ee ba a9	a.....
0040	b1 e6 d0 bd ae 1b 21 01	91 d6 45 09 90 8d ec 64	!....E...d
0050	00 f6 f3 92 a0 76 2e 3b	26 30 bb d1 a0 df 80 cf	v.; &0.....
0060	30 a6 21 c3 e1 38 8d 0f	d7 e7 95 f0 13 8a 48 15	0.!....8.....H...
0070	a3 f4 ec 03 5e 93 ae 9d	c0 3e 62 1c 29 a4 ab 77	A.....>b...w
0080	3c a7 0b a8 11 d7 53 6b	00 af cc 66 cc 86 81 a8	<.....Sk...F.....
0090	4e 74 c5 d6 2c 03 36 c4	e1 a5 52 e7 a5 e7 c8 b5	Nt.../.6...r.....
00a0	f1 ec b7 95 a9 2f fa 42	d6 a9 20 1c cd 32 36 72	/.B...26f...
00b0	54 2e d8 00 90 05 78 d0	dc 5f 57 da e1 a4 00 5c	T.....x...W.....

Payload is encrypted application data (tls.app_data), 250 bytes

Packets: 1484 · Dis

Encrypted payload

```
ubuntu@Ubuntu:~$ ps aux | head -1 && ps aux | grep dbus-sync | grep -v grep | sort -k6 -n | tail -1
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root          31218  0.1  1.3  35152 26560 ?        S      14:57   0:00 /usr/bin/dbus-sync
```

Problems Encountered / Things we learned

- Implant binary had to be compiled on Ubuntu to match target glibc version
- The initial exploit was supposed to be a KNOB attack, but produced visible activity
- How to chain multiple steps together (vulnerability -> exploit -> beachhead -> implant -> C2 -> exfiltration)
- Tradeoff between stealth and functionality in implant design - every capability added is a potential detection surface